UNED



MÁSTER

GESTIÓN Y ALMACENAMIENTO DE INFORMACIÓN NO ESTRUTURADA

TRABAJO PRÁCTICO:
BASES DE DATOS CLAVE-VALOR: REDIS



2020-2021

Dr. Juan Mario Haut Hurtado – Dr. Pablo Ruipérez García

Dr. Agustín C. Caminero Herráez – Dr. Luis Grau Fernández

MÁSTER UNIVERSITARIO EN INGENIERÍA Y CIENCIA DE DATOS

Tabla de contenido

1. Consideraciones iniciales: 3
2. Presentación del trabajo 4
3. Implementación 6
   3.1. Diseño de la base de datos 6
   3.2. Conjunto de datos 8
   3.3. Pruebas 9
4. Documentación a entregar 11

TRABAJO PRÁCTICO BASES DE DATOS CLAVE-VALOR: REDIS

1. Consideraciones iniciales:

En este documento se presenta el Trabajo Práctico (TP) del módulo "Bases de datos clave-valor. Redis", de la asignatura "GESTIÓN Y ALMACENAMIENTO DE INFORMACIÓN NO ESTRUCTURADA", del "MÁSTER UNIVERSITARIO EN INGENIERÍA Y CIENCIA DE DATOS" de la UNED. Este trabajo se realiza de forma individual.

- Se recomienda utilizar el Sistema Operativo Linux, en concreto la distribución Ubuntu.
- Se utilizará el lenguaje de programación Python 3 y en concreto la API que proporciona para trabajar con Redis (redis-py).
- Desarrollar los scripts de python utilizando notebooks de Jupyter, para que sea más sencilla la entrega, además del código desarrollado y cualquier comentario que el estudiante quiera añadir.
- Leer cuidadosamente las indicaciones del enunciado de la práctica en lo relativo a formato, orden de volcado de datos, etc., para que no haya errores en su implementación.
- Como siempre, para cualquier duda o comentario os podéis poner en contacto con el equipo docente utilizando los foros de la asignatura.

2. Presentación del trabajo

En este trabajo práctico se va a diseñar e implementar una base de datos clave-valor utilizando Redis para la gestión de un servicio de microblogging similar a Twitter. El objetivo es familiarizarse con la utilización de bases de datos clave-valor como Redis y comprobar que sus estructuras permiten construir modelos de datos efectivos para implementar muchos tipos de aplicaciones.

Nuestro "clon" de Twitter será estructuralmente sencillo, tendrá un funcionamiento eficiente, y podría ser distribuido entre un gran número de servidores con poco esfuerzo (aunque esto último queda fuera de los objetivos del presente trabajo práctico).

Como se ha comentado anteriormente, utilizaremos la librería para manejo de Redis que ofrece Python, denominada redis-py, gracias a la gran funcionalidad que ofrece para nuestros propósitos.

El primer paso para la realización de la práctica consistirá en levantar el entorno necesario para la utilización de una base de datos Redis, cuyo servidor opera sobre el puerto 6379. Para ello, se proporciona un archivo "docker-compose.yml"[1] para levantar el entorno. Es necesario instalar Docker-compose con el comando "sudo apt-get install docker-compose". Una vez hecho esto, ejecutar el comando "sudo docker-compose up –d" dentro de la carpeta donde hayamos copiado el archivo "docker-compose.yml". Este comando descargará e instalará Redis desde el repositorio de dockerhub, abriendo el puerto 6379 para realizar las conexiones y consultas contra la base de datos.

Una vez que tengamos levantada nuestra base de datos Redis utilizando docker-compose, será necesario instalar el paquete redis-py de Python. Para ello se ejecutará "pip install redis" en la consola o en el notebook de Jupyter que se utilice para desarrollar la práctica.

[1] El campus virtual UNED modifica el "-" por "_", por tanto, cuando se descargue el fichero de docker-compose.yml, aparecerá como docker_compose.yml. Simplemente cambiando "_" por "-", evitaremos errores durante el desarrollo de la práctica.

Posteriormente, la conexión a la base de datos se consigue utilizando el siguiente código:

```
import redis
redis_db=redis.Redis(host='127.0.0.1',port=6379, password='')
```

Como podemos observar, nos conectaremos a nuestra propia máquina (localhost) utilizando el puerto por defecto 6379. En nuestro caso, no hará falta incluir ninguna contraseña para realizar dicha conexión.

3. Implementación

El desarrollo del presente trabajo práctico se basa en tres partes diferenciadas. En la primera parte se implementará el diseño de la base de datos mediante una serie de funciones de Python que permitan la introducción de nuevos datos. En la segunda parte se poblará la base de datos previamente diseñada y finalmente, en la tercera parte se definirán funciones para realizar pruebas que permitan comprobar su correcto funcionamiento.

3.1. Diseño de la base de datos

Las funciones a implementar para construir nuestra base de datos clave-valor en Redis que imite el funcionamiento de Twitter son las siguientes:

- **nuevo_usuario**: Esta función recibirá el nombre del nuevo usuario y generará una nueva entrada en la base de datos utilizando un identificador incremental (proporcionado por Redis) para cada usuario. Este identificador incremental se puede utilizar para diferenciar cada una de las claves que contengan usuarios, de tal manera que dichas claves sean similares a la siguiente: "user:[id]", donde "id" es el identificador del usuario. Para cada usuario deberemos almacenar su nombre. De la misma forma se recomienda almacenar todos los usuarios y sus identificadores dentro de una misma estructura de datos única cuya clave puede ser "users".
- **nuevo_follower**: Una de las funcionalidades básicas de Twitter es la posibilidad de seguir a usuarios ya existentes. Para implementar esta funcionalidad crearemos la función "nuevo_follower", que recibirá el nombre de un usuario y el nombre del usuario que se convertirá en "follower" (seguidor) del primero, así como un "timestamp". Se recomienda utilizar nombres de clave similares a las utilizadas para los usuarios, de la forma "followers:[id]", donde "id" será el identificador del

usuario. Esta estructura de datos contendrá, para cada usuario, un conjunto de tuplas. En cada tupla se almacenará el identificador de un "follower" y el "timestamp" que representa el momento en el que le empezó a seguir.

- **nuevo_following**: Mediante esta función contraria a la función anterior, almacenaremos los usuarios seguidos por un usuario concreto. Recibe también un usuario original, el usuario a seguir y un "timestamp". La estructura de datos utilizada será similar, y su clave puede tener la forma "following:[id]", donde "id" es el identificador del usuario original. Se almacenarán igualmente para cada usuario, los identificadores de los usuarios a los que sigue y el momento en el que se les empezó a seguir.
- **seguir**: La función "seguir" recibirá un usuario original, un usuario a seguir, y un "timestamp" y hará uso de las dos funciones auxiliares anteriores para actualizar la información relativa al "follower" y a los "followings" del usuario original. Las funciones anteriores (nuevo_follower y nuevo_following) sólo podrán ser llamadas desde esta función, nunca de forma directa.
- **nuevo_post**: Esta función nos permitirá incluir nuevos mensajes en nuestra base de datos. Recibirá como parámetros el usuario que crea el mensaje, el cuerpo del mensaje y un "timestamp" representando el momento de creación del mensaje. Para cada mensaje se creará una clave cuyo nombre siga la estructura ya utilizada: "post:[idPost]", donde "idPost" será un contador incremental distinto al utilizado para los usuarios. Esta estructura contendrá, para cada post, el identificador del usuario que lo ha creado, el momento de su creación y el cuerpo del mensaje. Además, para una mejor gestión de los posts, se creará otra estructura "posts:[id]", donde "id" será un identificador de usuario. En esta estructura almacenaremos la lista de "idPost" de todos los posts pertenecientes a este usuario. **Es importante que esta lista no sólo almacene los posts creados por el usuario, sino también los creados por**

sus "followings" (usuarios a los que sigue), para una mejor gestión posterior del acceso a los posts.

Tanto para estas funciones como para las que se definirán en el apartado de "Pruebas", se puede añadir como parámetro adicional de la función la propia base de datos creada con la función mostrada en el apartado 2, o utilizar una única variable global dentro de las funciones para las llamadas a la misma. Se pide documentar claramente cuál de las dos opciones se utiliza.

3.2. Conjunto de datos

Una vez definidas las funciones que nos permiten implementar la base de datos propiamente dicha, poblaremos nuestra base de datos con un conjunto de datos definido previamente. Este conjunto de datos se proporciona al estudiante mediante dos ficheros CSV (*Comma Separated Values*): "twitter_sample.csv" y "relations.csv":

- El fichero "twitter_sample.csv" contiene un total de 111 tweets para cada uno de los cuáles se presenta el nombre del usuario que lo ha creado (columna User), el momento de la creación del tweet (Columna Post_Time) y el cuerpo del mensaje (Columna Tweet_Content). **Para este trabajo consideramos que los usuarios existen todos a la vez desde un primer momento, por lo que el primer paso que se pide para poblar la base de datos es la creación de todos los usuarios, mediante la función "nuevo_usuario". Es decir, no se almacena un momento de creación de la cuenta, aunque sí el momento en el que un usuario sigue a otro (importante para implementar las funciones de consulta).**
- El fichero "relations.csv" contiene un total de 22 relaciones entre usuarios, cada una de las cuáles contiene el usuario original (columna User), el usuario a quien sigue (columna Follows) y el momento en el que le sigue (columna Following_Time).

Se pide utilizar estos dos ficheros para, tras darles el formato necesario para nuestros intereses, poblar nuestra base de datos introduciendo inicialmente todos los usuarios utilizando la función "nuevo_usuario". A continuación, las relaciones entre ellos utilizando la función "seguir", y finalmente todos los tweets publicados utilizando la función "nuevo_post". Es importante tener en cuenta que las fechas y horas de los ficheros CSV no están en formato "timestamp" (float), sino que son fechas en formato texto con estructura "dd mmm aaaa hh:mm:ss", es decir, dos dígitos para el día, tres letras para el mes y 4 dígitos para el año, separados con espacios, y a continuación (tras un espacio adicional), dos dígitos para la hora, dos para los minutos y dos para los segundos, separados por el carácter ":". Será necesario pasar dicho formato a un formato legible por alguna librería de python dedicada al tratamiento del tiempo (por ejemplo "datetime") y después convertir la fecha completa en un "timestamp".

3.3. Pruebas

Una vez creada y poblada la base de datos, es el momento de realizar operaciones sobre la misma. Se pide la implementación de las siguientes 3 funciones:

- **obtener_followers**: Esta función recibirá un nombre de usuario y devolverá o imprimirá una lista con todos los nombres de los usuarios que le siguen, y en qué momento comenzaron a seguirle (en formato fecha, no "timestamp"), ordenados en el tiempo.
- **obtener_followings**: Esta función recibirá un nombre de usuario y devolverá o imprimirá una lista con todos los nombres de los usuarios a los que sigue, y en qué momento comenzó a seguirlos (en formato fecha, no "timestamp"), ordenados en el tiempo.
- **obtener_timeline**: Esta función recibirá un nombre de usuario y un parámetro booleano denominado "tweets_propios". Devolverá o imprimirá por pantalla todos los tweets correspondientes a dicho usuario

(propios o publicados por los usuarios a los que sigue), ordenados por fecha de publicación del tweet. **Deberá utilizarse obligatoriamente la función "SORT" proporcionada por Redis dentro de esta función, para generar la ordenación de los tweets.** Además, sólo se mostrarán aquellos tweets de los usuarios seguidos por el usuario original que sean posteriores al momento en el que el usuario original comenzó a seguirles. El parámetro booleano "tweets_propios" permitirá indicar si queremos mostrar tanto los tweets propios como los de los usuarios seguidos (tweets_propios = True), o sólo los de los usuarios seguidos (tweets_propios = False). La salida exacta será el nombre del usuario que escribió el tweet, el cuerpo del tweet y el momento de publicación (en formato fecha, no "timestamp")

No es necesario (aunque se puede hacer) incluir celdas en el notebook con las pruebas de estas funciones (únicamente documentarlas correctamente sería suficiente), ya que el equipo docente ejecutará una batería de pruebas sobre las tres funciones para comprobar su correcto funcionamiento.

4. Documentación a entregar

Se entregará un archivo comprimido en zip denominado "TP-Redis-ApellidosNombre.zip", donde *Apellidos* y *Nombre* han de sustituirse por los valores correspondientes del estudiante. Al descomprimir, dentro de la carpeta raíz deberá haber uno o dos archivos, en función de si el estudiante decide incluir la documentación de la práctica en un fichero pdf aparte, o en el propio notebook de Jupyter que contiene el código:

Notebook Jupyter:

Un notebook de Jupyter denominado redis_twitter.ipynb contendrá todo el código necesario para conectarse a la base de datos Redis, así como todas las funciones especificadas en el enunciado y el código necesario para volcar los datos a la base de datos diseñada. **No es necesario volver a entregar los ficheros de datos CSV, ya que el equipo docente probará la práctica copiando dichos CSV en la misma carpeta en la que se encuentra el fichero.ipynb.**

Se pide incluir también el nombre y los apellidos del estudiante en el Notebook para evitar errores.

Se pide entregar los notebooks sin ningún "output" resultado de pruebas realizadas antes de la entrega, ya que se ejecutarán por completo para probarlos. No es necesario incluir pruebas con las llamadas a las funciones de prueba (obtener_followers, obtener_followings y obtener_timeline), ya que el equipo docente ejecutará una batería de pruebas.

Documentación:

Como se ha indicado anteriormente, la documentación se puede entregar en formato PDF o como celdas de texto dentro del propio notebook en el que se encuentra el código. De cualquiera de las dos maneras se documentarán todas las decisiones de diseño tomadas por el estudiante a la hora de

implementar el trabajo práctico. Se valorará positivamente una expresión escrita correcta y estructurada, así como el uso de tablas y diagramas explicando las estructuras de datos utilizadas, la arquitectura de la aplicación, etc. En caso de entregar un fichero PDF, el nombre de este será "Memoria_Redis_ApellidosNombre.pdf".

Se valorará positivamente un comentario final sobre las impresiones del estudiante acerca de este trabajo práctico, así como las dificultades encontradas.

Importante: La puntuación de este trabajo práctico no atenderá únicamente al correcto funcionamiento de la solución entregada, sino también a su eficiencia desde el punto de vista de las estructuras de datos utilizadas y las decisiones de diseño tomadas. De la misma manera, también se atenderá a la existencia de una documentación lo suficientemente completa, correctamente escrita y explicativa de todos los aspectos que el estudiante considere importantes (decisiones sobre el código, dificultades, opiniones, etc.). Como guía, el correcto funcionamiento de la aplicación permitirá obtener 7 puntos, y los 3 restantes se dividirán entre la calidad de la documentación entregada y las decisiones de diseño tomadas.